Software Quality Journal (2025) 33:30
https://doi.org/10.1007/s11219-025-09728-1

RESEARCH

ReAPR: Automatic program repair via retrieval-augmented large language models

Zixin Liu[1] · Xiaozhi Du[2] · Hairui Liu[1]

Accepted: 14 July 2025 / Published online: 22 July 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Automatic Program Repair (APR) aims to automatically fix software defects, significantly reducing the efforts of manual debugging. Recent studies have demonstrated impressive results in utilizing Large Language Models (LLMs) for software bug fixing. Current LLM-based approaches depend solely on the pre-trained knowledge of LLMs, overlooking the prior knowledge contained in historical bug repair records, which increases the likelihood of hallucinations. To address this challenge, this paper proposes ReAPR, a retrieval-augmented framework for APR. We first curate a high-quality retrieval database by carefully compiling and filtering the existing datasets for APR. Subsequently, ReAPR leverages a retriever to fetch bug-fix pairs similar to the target bug from a retrieval database, providing contextual hints to guide the LLMs in the repair process. We then investigate two techniques to retrieve bug-fix pairs associated with the function to be fixed: BM25 and Dense Passage Retrieval (DPR). After retrieving the relevant bug-fix pair, we construct a prompt and integrate the retrieved pair into it. Besides, we also compare the proposed RAG-based approach with the parameter-efficient fine-tuning (PEFT) approaches on repair performance. To validate the effectiveness of ReAPR, we conduct extensive experiments based on the widely-used benchmark dataset Defects4j 2.0 as well as the latest benchmark GitBug-Java. The results show that ReAPR, based on the CodeLlama(7B) backbone, successfully fixes 68 and 59 bugs in the DPR and BM25 settings, respectively, in Defects4j 2.0, outperforming the best baseline approach by 18 and 9 bugs under the same repair settings.

Keywords Automated Program Repair · Retrieval-Augmented Generation · Large Language Models · Prompt Learning

✉ Xiaozhi Du
xzdu@xjtu.edu.cn

Zixin Liu
ZixinLiu@stu.xjtu.edu.cn

Hairui Liu
1153739364@stu.xjtu.edu.cn

1 School of Software Engineering, Xi'an Jiaotong University, No. 28 Xianning West Road, Xi'an 710049, Shaanxi, China

2 Shaanxi Joint (Key) Laboratory for Artificial Intelligence, School of Software Engineering, Xi'an Jiaotong University, No. 28 Xianning West Road, Xi'an 710049, Shaanxi, China

1 Introduction

With the growing reliance on software in both production and everyday life, software bugs have become an increasingly critical issue that can not be ignored. These bugs can lead to severe security issues (Richards, 1990) and financial losses (Matteson, 2018), making the time and effort dedicated to fixing them a significant and expanding part of the software development process. Automated Program Repair (APR), which focuses on automatically fixing software bugs, has garnered increasing attention as a means to reduce the workload of bug fixing and enhance software quality.

Existing works for APR can be categorized into two groups: template-based (Just et al., 2014; Li et al., 2022; Harer et al., 2018; Li et al., 2020; Le et al., 2019) and deep-learning-based (Tan & Roychoudhury, 2015; Chen et al., 2017; Weimer et al., 2009; Long & Rinard, 2015). Template-based methods propose using predefined program fix templates (also known as "fix patterns" or "transformation schemas") to generate repair patches. However, these repair templates are typically manually designed by experts, and therefore cover only a limited range of bug types. In contrast, deep learning-based methods approach the repair task as a translation problem, employing a Neural Machine Translation (NMT Tufano et al. (2019)) model to translate buggy statements into correct ones. Specifically, they extract buggy code from the open-source software community and correct it to create Bug-Fix Pairs (BFPs). The NMT model then learns repair knowledge from these BFPs to perform program repair. However, the scarcity of high-quality BFPs limits the effectiveness of deep learning-based methods.

Recently, inspired by the significant advancements in text generation by Large Language Models (LLMs), various studies (Xia & Zhang, 2022; Kolak et al., 2022; Prenner et al., 2022) have demonstrated the effectiveness of LLMs for APR, when compared with existing APR tools. For instance, the InCoder model can fix 72% more bugs than the traditional and learning-based APR techniques (Jiang et al., 2023). While LLMs have achieved impressive results in bug fixing, current LLM-based methods rely exclusively on the pre-trained knowledge of these models, without leveraging the valuable insights from historical bug repair records. This lack of integration with prior knowledge increases the risk of generating hallucinations or incorrect fixes, which can undermine the reliability of these approaches. To address this limitation, there is a growing need for methodologies that combine the inherent capabilities of LLMs with historical bug data to enhance the accuracy and effectiveness of bug repair systems.

Retrieval-Augmented Generation (RAG) (Lewis et al., 2020; Jiang et al., 2023; Li et al., 2022) is a state-of-the-art technique designed to overcome the limitations of LLMs by enhancing them with external knowledge retrieved from a knowledge database. This approach allows LLMs to access relevant information that may not be present in their training data, thereby improving their accuracy and relevance in generating responses. RAG consists of three key components: a knowledge database, a retriever, and an LLM. The knowledge database is a vast repository of texts collected from various reliable sources, such as Wikipedia, scientific articles, and news archives. The retriever uses a text encoder, such as BERT (Devlin, 2018), to compute embedding vectors for each text, effectively transforming the text into a format that can be easily compared and retrieved based on semantic similarity. Once the retriever identifies the top k most relevant texts, these are provided as context to the LLM, which then generates a response to the given query or question. By integrating this retrieved external knowledge, RAG enables LLMs to produce more informed and contextually accurate answers, enhancing their performance in various tasks.

In this paper, we present ReAPR, a retrieval-augmented APR framework that focuses on complete Java function repair. The process of bug fixing by ReAPR can be divided into a two-

step process: retrieval and repair. In the first step, given a bug function to be fixed, the retrieval module uses the BM25 algorithm (sparse retrieval) and the DPR algorithm (dense retrieval) to retrieve the bug-fix-pair related to it from the constructed external knowledge base. In the second step, we choose six of the most popular generative LLMs hosted on Hugging Face platform (Transformer Vaswani et al., 2017 model with only decoder structure) as our generators. Among them, three belong to the GPT-Neo series, differing only in parameter sizes. Overall, the models' parameter sizes range from 125M to 7B to verify the impact of different model sizes on repair performance. We then feed the bug-fix-pair, retrieved from the first step, into the already constructed prompt, and then use the entire prompt as input to LLMs to generate a fix for the full Java function. We evaluate our ReAPR on the benchmark dataset Defects4j 2.0 (Just et al., 2014) and GitBug-Java (Silva et al., 2024), in Java programming languages. Due to the absence of bug-fix-pairs related to GitBug-Java in the retrieval corpus, ReAPR performs poorly on the GitBug-Java benchmark. In contrast, the retrieval corpus contains relevant bug-fix-pairs for Defects4j 2.0, enabling ReAPR to achieve state-of-the-art performance on this dataset. The experimental results on these two benchmarks demonstrate that leveraging bug-fix-pairs retrieved by our retriever, which are related to the bugs to be fixed, significantly facilitates effective bug repair.

To summarize, the primary contributions of this paper are as follows:

- **New Dataset.** We curate a high-quality retrieval database by carefully compiling and filtering the existing datasets for APR.
- **Novel Approach.** We propose a retrieval-augmented method to retrieve related bug-fix-pairs to assist the APR task using sparse and dense retriever, respectively.
- **Extensive Experiments.** We conduct extensive experiments on the widely used Defects4j 2.0 benchmark and the latest benchmark GitBug-Java to evaluate the effectiveness of our approach.
- **Open-Sourced Experimental Code.** Reproducibility is an important aspect of research. To make it easier for others to replicate our experiments, we have open-sourced our code on GitHub. The repository is available at: https://github.com/ZXLiu/ReAPR

The remainder of this paper is organized as follows. Section 2 describes the background knowledge relevant to this paper. Section 3 presents the specific methodology of our work and its details. Section 4 gives a detailed introduction to the specific process of our experiment as well as related details. Section 5 provides a comprehensive and systematic analysis of the experimental results. Section 6 presents some work related to this paper. Section 7 presents some limitations of this paper. Section 8 concludes the paper as a whole.

2 Background

In this section, we introduce several background knowledge, including the APR and LLMs, that to facilitate the better understanding our of work.

2.1 Automatic Program Repair (APR)

Automated Program Repair (APR) tools are designed to generate patched code based on the original code and the identified buggy location. In practice, a patch generated by an APR tool is validated against the test suite and deemed plausible if it passes all tests; however, passing the test suite does not guarantee the patch's correctness. In fact, correct patches are those that are not only plausible but also semantically accurate, effectively addressing the underlying

bug. Typically, the general workflow of APR tools for fixing program bugs consists of three steps, as illustrated in the dashed box in Fig. 1: (1) Fault localization, (2) Patch generation, and (3) Patch validation. The goal of fault localization is to identify and pinpoint defects in the program, enabling APR tools to apply precise fixes. Once specific defect locations are identified, the APR tool employs patch generation techniques to create candidate fixes. These patches can either be literal program patches or concrete proposals to guide the repair process. After generating candidate patches, it is crucial to utilize the test suite and other tools to evaluate and filter these candidates, ultimately identifying plausible and correct patches.

2.2 Large Language Models (LLMs)

LLMs have become ubiquitous in NLP, achieving impressive performance on many tasks such as machine translation (Sutskever et al., 2014), text summarization (Liu, 2019), and classification (Yang et al., 2019). LLMs follows the Transformer architecture (Vaswani et al., 2017) - an encoder for capturing input representations and a decoder for generating output tokens. From the perspective of neural architectures, LLMs can be divided into encoder-only, decoder-only, and encoder-decoder models. Encoder-only models (such as BERT Devlin (2018)) contain only the encoder components of a Transformer. They are typically designed to learn data representations and are trained using the Masked Language Modeling (MLM) objective – a small percentage (e.g., 15%) of tokens in the training data will be replaced by masked tokens, and then the models are trained to predict the original values of the masked tokens based on the bidirectional contexts. Decoder-only models (such as GPT-3 (Brown, 2020)) are large generative models that use the decoder to predict the next token output given all previous tokens (i.e., left context or prefix only). To combine the usage of both encoder and decoder, encoder-decoder models (such as T5 (Raffel et al., 2020) and BART (Lewis et al., 2019)) have also been proposed for sequence-to-sequence tasks where the training objective aims to recover the correct output sequence given the original input (e.g., corrupted to uncorrupted). In our work, we follow the decoder-only models such as GPT for program repair.

3 Approach

Our proposed retrieve-enhanced APR framework, ReAPR, guides LLM to fix a bug better by retrieving its related bug-fix-pair to the bug to be fixed. In this section, we first introduce the overall architecture of ReAPR, and then elaborate each component of that.

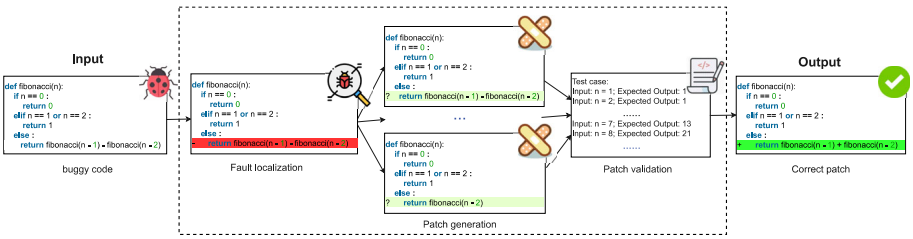


Fig. 1 APR general process

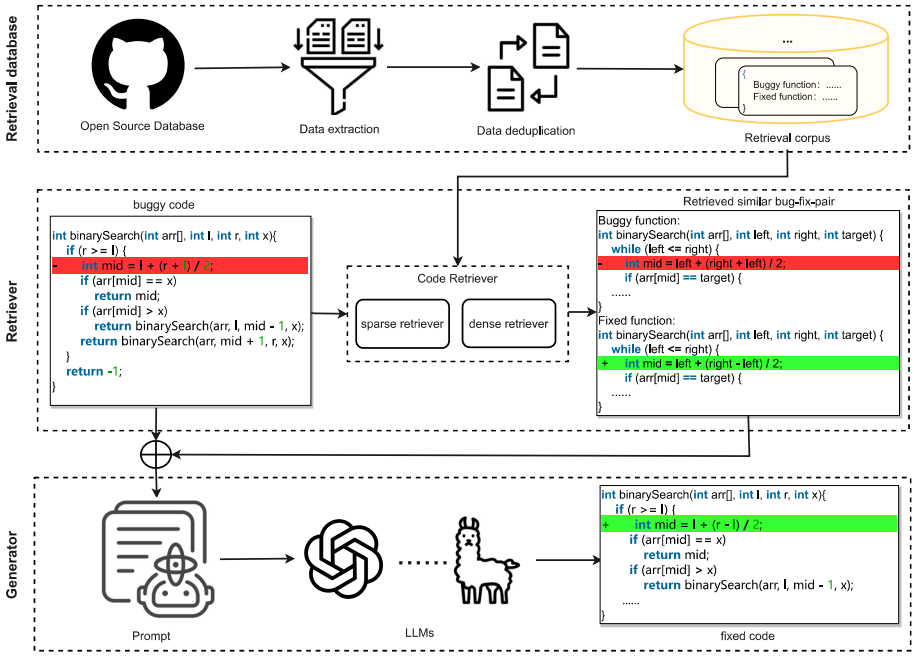


Fig. 2 An illustration of the ReAPR framework. Given a buggy code snippet to fix, ReAPR first retrieves the bug-fix-pair associated with it from the retrieval corpus. Then, it is concatenated with the code to be repaired to form prompt as input to the generator, the repaired code will be generated based on it

3.1 Overall architecture

Our proposed ReAPR can be divided into three parts: (a) retrieval database construction, (b) retriever, and (c) fixed code generator. The retriever module consists of a sparse retriever and a dense retriever. The sparse retriever is based on the BM25 (Robertson et al., 2009) algorithm. Here, we use the BM25 algorithm that has been encapsulated in ElasticSearch[1] to realize sparse retrieval. The dense retriever is built on the basis of the DPR (Karpukhin et al., 2020) model and uses bidirectional Transformer encoder to encode buggy code and its corresponding fixed code respectively. Here, we use GraphCodeBERT (Guo et al., 2020) to initialize our encoder and contrastive learning is used for training fine-tuning. The retrieval module is described in detail in Sec. 3.3. In order to integrate the relevant information obtained from the retrieval module, the generator module first encapsulates the retrieved bug-fix-pair together with our buggy code to be fixed into a prompt, which is used as the input of the model. We choose LLMs with parameter sizes ranging from 125M to 7B as the backbone models to generate the complete function as a repair result, as detailed in Sec. 3.4. The overall framework of ReAPR is shown in Fig. 2.

3.2 Retrieval database construction

To retrieve the bug-fix pair related to the bug to be addressed, we first build a retrieval database. Since we focus on fixing bugs in complete Java functions, each entry in our database must

[1] Elasticsearch: https://github.com/elastic/elasticsearch

represent a fixed instance of a complete function. With this in mind, we thoroughly investigate the existing datasets utilized in previous work (Chen & Monperrus, 2018; Tufano et al., 2019; Monperrus et al., 2021; Jiang et al., 2023), which include complete Java functions along with their corresponding fixes, as summarized in Table 1. After merging them one by one, we get a total of 397,933 bug-fix-pairs. Since the bug-fix pairs primarily originate from real GitHub commits and are collected using similar strategies that often focus on the most popular projects, duplicates are inevitably included. To this end, we deduplicate the bug-fix-pairs collected from each dataset, and remove the records with the same bug and its corresponding fix as well as the bugs to be fixed in the benchmark dataset. Finally, we obtain 364,063 bug-fix-pairs.

3.3 Retriever

Sparse retriever Sparse retrieval methods, also known as term-based retrieval models, are designed to evaluate the relevance of documents based on the presence and frequency of query terms within them. Specifically, after a document is segmented into words, the frequency of each word is counted, which is the term frequency TF. Then, the ratio of the number of documents in which the word appears to the total number of documents is counted according to each word, which is the inverse document frequency IDF. The TF and IDF are multiplied to get the importance of the term, and the vector of the importance of the term for the current document is formed, and the similarity of the vectors between the two documents is the similarity of the documents. BM25 (Robertson et al., 2009) is a classical sparse retriever that employs a probabilistic model to measure the relevance between search terms and documents. It does this by considering the term frequency (TF) and inverse document frequency (IDF) of query terms within the documents. In our scenario, BM25 treats each code fragment as a sequence of code tokens, using a bag-of-words representation. The match score computed by BM25 reflects the lexical similarity between the query and the document, which is then used to rank the documents based on their relevance.

Dense retriever Dense retrievers, in contrast, are designed to capture the semantic similarity between queries and documents by mapping them into a dense, continuous vector space. Unlike BM25, which focuses on matching terms lexically, dense retrievers leverage deep learning models to encode both queries and documents into high-dimensional vectors. Relevance is then assessed by measuring the similarity between these vectors, commonly using metrics such as cosine similarity or dot product.

In this paper, we follow the Dense Passage Retriever (DPR) (Karpukhin et al., 2020), which aims to utilizes a dual-encoder model, where one encoder is used to encode the query and another to encode the passages, allowing for more effective matching based on semantic similarity. Specifically, we adopt two Transformer-based bidirectional encoders E_B and E_F,

Table 1 Constituent sources of the retrieval database

Name	Type	#Bugs	Languages
CodRep Chen and Monperrus (2018)	single-line patches	58,069	Java
BFP Tufano et al. (2019)	real-world bugs	58,350/65,455	Java
MegaDiff Monperrus et al. (2021)	real world bugs full-file diffs	72,393	Java
Recoder Jiang et al. (2023)	single-hunk patches	143,666	Java

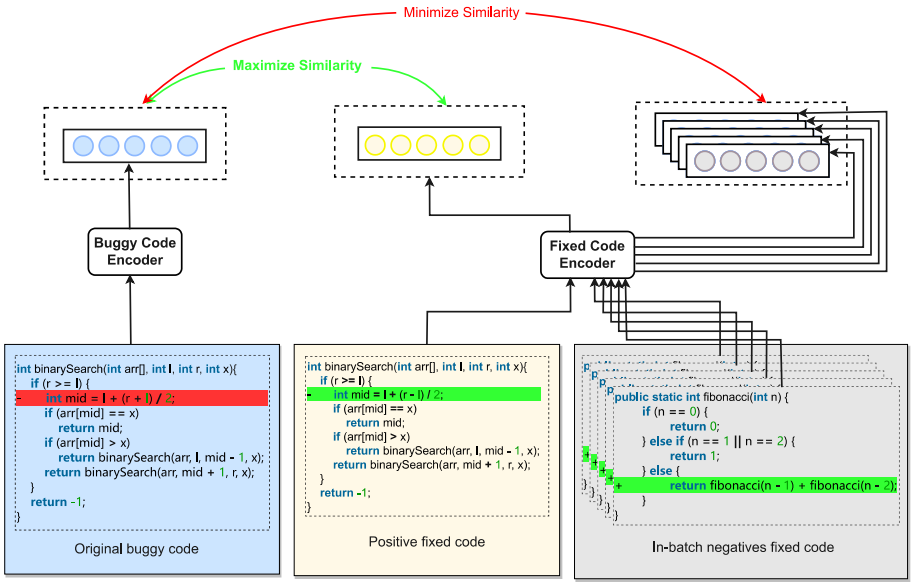


Fig. 3 Illustration on the training process of the retriever in our proposed framework ReAPR

to embed the buggy code and fixed code into vectors, respectively. As the data of our retrieval database is composed of the form of bug-fix-pairs, we encode the buggy code snippets from the retrieval database using E_B and build indices for them. The corresponding bug-fix code snippets from the retrieval database are encoded using E_F. We use the representation of the `[CLS]` token as the vector for the entire code fragment after it has been embedded by the encoder, where `[CLS]` represents the initial token of the code snippet. We then compute the similarity using the inner product of these vectors, with the similarity calculation formula as follows:

$$sim\,(b, f) = E_B\,(b)^T\, E_F\,(f) \tag{1}$$

In the training phase, we utilize batch negatives to compute the contrastive loss using InfoNCE (Oord et al., 2018), as this approach enhances the model's ability to learn discriminative representations. By leveraging batch negatives, we can effectively compare each positive sample against multiple negative samples within the same batch, which helps the model identify subtle differences between similar samples. The InfoNCE (Oord et al., 2018) loss function specifically exploits the relationships among in-batch samples, treating other samples as potential negatives for a given query. This not only increases the robustness of the training process but also mitigates the impact of false negatives commonly encountered in large datasets. Therefore, combining batch negatives with InfoNCE (Oord et al., 2018) facilitates a more effective learning process, ultimately improving the performance of code retrieval. In particular, the InfoNCE (Oord et al., 2018) loss function is formulated as follows:

$$L\left(b, f^+, f_1^-, f_2^-, \ldots, f_m^-\right) = -\log \frac{e^{sim(b, f^+)}}{e^{sim(b, f^+)} + \sum_{i=1}^{m} e^{sim(b, f_i^-)}} \tag{2}$$

Since both E_B and E_F take source code as input, we chose to initialize the encoder using GraphCodeBERT (Guo et al., 2020), given its pre-training on code snippets and strong understanding of code semantics.

For each training instance (b_i, f_i), the corresponding fix f_i is considered positive, and the fixes $f_1,\ldots, f_{i-1}, f_{i+1},\ldots,f_{bsz}$ are negative. We then use the Buggy Code Encoder to embed b_i as a vector and the Fixed Code Encoder to embed $f_1,\ldots, f_{i-1}, f_{i+1},\ldots,f_{bsz}$ as vectors. Finally, we employ the InfoNCE loss function for training, aiming to maximize the similarity between b_i and f_i, while minimizing the similarity between b_i and $f_1,\ldots, f_{i-1}, f_{i+1},\ldots,f_{bsz}$. Figure 3 shows an example of the fine-tuning of the APR task by ReAPR's dense retriever module.

3.4 Generator

In this paper, we focus our APR task on complete functions. Specifically, we take a full function body that contains a bug and aim to produce the corresponding full function body with the fix. However, it is unreasonable to input the entire body of the function into LLMs, based on the following two reasons. Firstly, LLM is not trained on APR (the model does not know that the goal is to generate its corresponding fixed function). Secondly, it does not make full use of the bug-fix-pair information related to it obtained from the retriever. Based on these reasons, we construct a prompt, as shown in Fig. 4, providing 2 pairs of bugs and their corresponding fixes, following (Xia et al., 2023). The first pair of examples is used to integrate the bug-fix-pairs retrieved from the retrievers, the second pair of examples is from the project where the target bug is located, and finally the bug to be fixed, so that LLM can do a small amount of learning. This allows LLM to recognize the task, while also making full use of the retrieved information, which can better generate a complete repair function. We also designed a no-retrieval prompt, as shown in Fig. 5, to facilitate subsequent experiments and enable a comparative assessment of the repair performance between the two approaches.

With prompt in place, we use it as input to the LLM. Since we are going to do a full body repair, we choose 6 popular generative LLMs hosted on Hugging Face (Pepe et al., 2024) platform, among which three belong to the GPT-Neo series and differ only in their parameter sizes. The parameter sizes of these six generative LLMs range from 125M to 7B. Table 2 summarizes the specific details of the generative LLMs.

- **GPT-Neo** (Black et al., 2021). This model is open-source implementations of the GPT-3 transformer architecture. In our experiments, we use GPT-Neo models with 125M, 1.3B

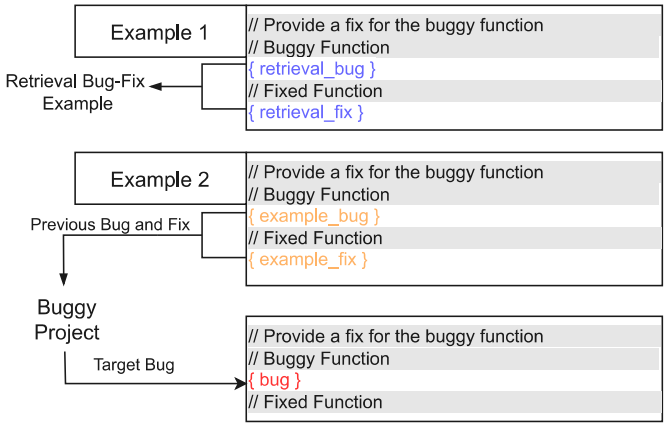


Fig. 4 APR prompt with retrieval

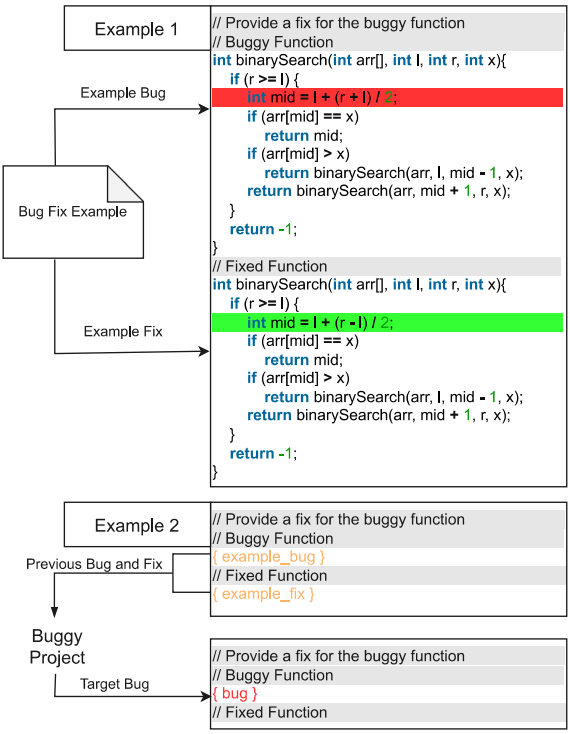


Fig. 5 APR prompt without retrieval

and 2.7B parameters. This model was trained on The Pile, a dataset combining 22 diverse text-based datasets with 7.6% containing open-source GitHub code.

- **Phi-2** (Javaheripi et al., 2023). This model is an LLM with 2.7B parameters developed by Microsoft Research. Despite its small size, it achieves comparable performance to other larger-scale models thanks to its pre-training on “textbook quality" data. Even outperforming some larger models on some tasks.
- **DeepSeekCoder** (Guo et al., 2024). This model is an LLM with 6.7B parameters developed by DeepSeek Research. Deepseek Coder is pre-trained on project-level code corpus by employing a window size of 16K and a extra fill-in-the-blank task, to support project-level code completion and infilling. For coding capabilities, Deepseek Coder achieves state-of-the-art performance among open-source code models on multiple programming languages and various benchmarks.

Table 2 Studied LLMs. (N.R.: Not Released)

Model	#Parameters	Training Dataset	Type
GPT-Neo	125M/1.3B/2.7B	The Pile	Generative
Phi-2	2.7B	N.R.	Generative
DeepSeekCoder	6.7B	N.R.	Generative&Infilling
CodeLlama	7B	N.R.	Generative&Infilling

- **CodeLlama** (Roziere et al., 2023). CodeLlama belongs to the Llama2 family (Touvron et al., 2023) and is specifically trained for code tasks. The 7B parameters version of CodeLlama shows state-of-the-art performance on most downstream tasks compared to models of the same size. It is trained with a context window of 4,096 tokens and uses multi-query attention (Shazeer, 2019).

We use nucleus sampling (Holtzman et al., 2019) with a sampling temperature. The choice of temperature often leads to different results, and for a problem like APR, it's not obvious how to pick an optimal temperature value. For some bugs, one may prefer a lower temperature value in order to reach a reasonable patch quickly. For harder bugs, higher temperature values can be used to generate more unique patches in an attempt to provide a fix. In our experiments, we use the default settings used in previous works (Chen et al., 2021; Fried et al., 2022) in order to compare with the original method and verify the effectiveness of our method.

4 Experiment setup

To demonstrate the effectiveness of our approach, we conduct a comprehensive study and analysis on the repair of complete Java functions on the Defects4j (Just et al., 2014) benchmark. Specifically, we aim to answer the following Research Questions (RQs):

RQ1: How does ReAPR perform on the APR task when compared with other baselines, under different retrieval settings?
RQ2: How does the quality (patch correct rate) of patches generated by ReAPR under different retrieval settings?
RQ3: How to trade off generation efficiency and patch correct rate of ReAPR?
RQ4: How do different numbers of bug-fix-pairs affect the repair performance of ReAPR?
RQ5: How do RAG and PEFT perform in automatic program repair?

Finally, we also perform a case study to qualitatively analyze how does ReAPR work.

4.1 Datasets

To validate the effectiveness of our proposed approach, we adopt Defects4j (Just et al., 2014) and GitBug-Java (Silva et al., 2024). Defects4j has been widely recognized as the preeminent benchmark in APR. Given its recent updates incorporating additional bugs from new projects, we utilize the latest Defects4j 2.0 version in our study. The original Defects4j 1.2 encompasses 391 bugs (after removing 4 deprecated cases) sourced from 6 open-source Java projects. The more recent Defects4j 2.0 builds upon Defects4j 1.2 by introducing 438 new bugs from 9 additional projects. These bugs are curated from prominent, large-scale Java projects, such as Apache Commons libraries, all of which boast comprehensive test suites. GitBug-Java is the latest benchmark for Java bug fixes, featuring 199 bugs extracted from the 2023 commit histories of 55 well-known open-source repositories. Since it collects recent bugs, this benchmark suffers less from data leakage issues compared to older datasets.

Our research focuses specifically on repairing bugs within complete Java functions. To this end, we extract the entire Java functions containing bugs from all projects in both Defects4j 2.0 and GitBug-Java. This approach yields a total of 573 bugs for our repair efforts, including 483 bugs extracted from the 835 bugs in Defects4j 2.0 and 90 bugs extracted from the 199 bugs in

Table 3 The number of complete function bugs in Defects4j 2.0

Project Name	Total Number	Complete Function Number
Chart	26	16
Cli	39	23
Closure	174	105
Codec	18	11
Collections	4	1
Compress	47	33
Csv	16	11
Gson	18	9
JacksonCore	26	13
JacksonDatabind	112	51
JacksonXml	6	4
Jsoup	93	53
JxPath	22	7
Lang	64	42
Math	106	72
Mockito	38	16
Time	26	16
	835	**483**

GitBug-Java. This dataset selection offers several advantages: (1) It provides a diverse range of real-world bugs from well-established Java projects. (2) The inclusion of extensive test suites facilitates thorough validation of our repair techniques. (3) By considering Defects4j 2.0 and GitBug-Java, we ensure our approach is tested on both established and more recent bug patterns. Table 3 and Table 4 summarize the specific details of the dataset.

4.2 Baselines

We select several state-of-the-art tools in the APR field, including both learning-based and template-based approaches, as our baselines. Comparing ReAPR with these baselines demonstrates its effectiveness more convincingly. For learning-based APR, we chose Recoder (Zhu et al., 2021) and DLFix (Li et al., 2020). Recoder is considered the best-performing learning-based APR tool. These tools utilize Neural Machine Translation (NMT Tufano et al. (2019)) models to generate patches based on the given buggy line and its surrounding context. For template-based APR, we select TBar (Liu et al., 2019a) and AVATAR (Liu et al., 2019b). TBar is regarded as the best-performing template-based APR tool. These approaches leverage predefined templates to fix corresponding errors. Additionally, we include AlphaRepair (Xia & Zhang, 2022), which directly employs LLMs like CodeBERT for program repair.

4.3 Implementation

Retriever We implement sparse and dense retrievers in the retrieval module to explore the effectiveness of different retrieval methods for the APR task. The sparse retriever directly uses the robust BM25 (Robertson et al., 2009) algorithm implemented by the ElasticSearch search engine. This mature approach provides efficient word-based matching, exploiting the lexical

Table 4 The number of complete function bugs in GitBug-Java

Project Name	Total Number	Complete Function Number
AuthMe-ConfigMe	2	1
LMAX-Exchange-Simple-DSL	2	1
TheAlgorithms-Java	4	1
aws-aws-secretsmanager-jdbc	1	1
cloudsimplus-cloudsimplus	1	1
crawler-commons-crawler-commons	1	1
crowdin-crowdin-api-client-java	1	1
dmak-jaxb-xew-plugin	1	1
fishercoder1534-Leetcode	1	1
fusesource-jansi	1	1
jhy-jsoup	29	14
jitterted-ensembler	6	4
mthmulders-mcs	3	1
nikoo28-java-solutions	1	1
retel-io-ari-proxy	1	1
revelc-formatter-maven-plugin	1	1
semver4j-semver4j	3	1
snowflakedb-snowflake-jdbc	4	1
spring-projects-spring-retry	2	1
stellar-java-stellar-sdk	3	2
traccar-traccar	81	51
vmzakharov-dataframe-ec	2	1
xtremexp-UT4X-Converter	2	1
	153	**90**

similarity between the query and the potential repair. On the other hand, the dense retriever uses the more semantically understanding DPR algorithm, and we use GraphCodeBERT (Guo et al., 2020) for model weight initialization and contrastive learning for training fine-tuning on the dataset. In this work, we use the retrieval database mentioned in Section 3.2 as the training data set of the model, and different from the retrieval database, we directly use all the data, a total of 365,391 bug-fix-pairs for training. We use in-batch negative values and perform distributed training on 4 NVIDIA RTX 3090 GPUs with a batch size of 32 per card, a total batch size of 128, and a learning rate of 5e-5 for a total of 30 epochs of training.

Fixed-code generator We carefully selected five generative models hosted on the Hugging Face platform, with parameter sizes ranging from 125M to 7B, to explore the understanding of different sizes of models for retrieved bug-fix-pairs and the ability of programs to be automatically fixed. In our implementation, we directly utilize the pre-trained weights of each model, while to ensure consistency with previous LLM-based program repair studies, we adopt nucleus sampling (Holtzman et al., 2019) with top-p-value of 0.95 and temperature of 0.8. This approach balances the diversity and quality of generated repairs while introducing controllable randomness. For each bug, we generate multiple fix candidates -specifically 20, 50, 100, 150, and 200 samples -allowing us to evaluate the impact of the number of generation on the fix quality and success rate. This generation setting is not only consistent with previous

work on LLMs (Kolak et al., 2022; Chen et al., 2021; Fried et al., 2022), but also allows us to study the efficiency-effectiveness trade-off in the repair process, providing valuable insights for optimizing APR systems.

4.4 Evaluation metrics

To comprehensively evaluate repair performance, we employ a rigorous set of two-factor verification processes. First, we utilize Defects4j benchmark suite for automated testing of each generated patch. For each bug, we run all the test cases of the bug in Defects4j, effectively filtering out the patches that fail to pass all the tests, so as to obtain a set of ***plausible patches*** that can pass all the test cases. However, passing all test cases is not sufficient to guarantee the complete correctness of a patch, since a plausible patch is not necessarily semantically correct. Therefore, to determine the ***correct patches***, we follow the standard practice in the APR research field and perform a meticulous manual inspection of each plausible patch to determine whether it is syntactically or semantically equivalent to the reference patch provided by the developer.

5 Experimental results and analysis

5.1 RQ1: Overall performance with different retrieval settings

Comparison of ReAPR with the SOTA method To better demonstrate the effectiveness of our proposed ReAPR, we compared its best results under three retrieval settings with other SOTA tools in the APR domain, as shown in Table 5. The results reveal that our no-retrieval setting's best repair performance (Correct/Plausible = 47/108) is already comparable to current SOTA tools. Moreover, the best repair performances with retrieval (Sparse Retrieval: 59/113, Dense Retrieval: 68/130) surpass current SOTA tools in both correct and plausible patches. This comparison clearly illustrates the superior performance of ReAPR across different retrieval settings, particularly highlighting the significant improvements achieved through sparse and dense retrieval methods.

Repair results without retrieval The row "ReAPR(No Retrieval)" in Table 5 shows the results of the six generative models in the no-retrieval setting. The two integers in each cell represent the number of plausible and correct patches, respectively. We first observe that there is a scaling effect on repair effectiveness. As we increase the size of the model, we also increase the number of plausible and correct patches generated. We observe that groups of GPT models trained on the same dataset consistently improve in performance as larger models are used on the benchmark dataset. However, it is interesting to observe that the two models, GPT-Neo(2.7B) and Phi-2(2.7B) with the same model size, are significantly different in their repair effects. The Phi-2(2.7B) model is much better than GPT-Neo(2.7B) model in both the number of plausible patches and the number of correct patches. We argue that this is due to the fact that the Phi-2(2.7B) model is trained using "textbook level" high-quality data specifically designed to teach reasoning, knowledge, and common sense. Curated data allows the model to learn more from less information. In addition, we found that DeepSeekCoder(6.7B) and CodeLlama(7B) outperform other generative models by a wide margin, not only because of their large number of parameters, but also because they have been trained and optimized specifically for code-related tasks.

Table 5 Complete function repair results under different methods

Method		**Accuracy**				
AlphaRepair		50/90				
Recoder		49/96				
DLFix		30/65				
TBar		42/81				
AVATAR		27/53				
	Base Model	Patch Generation Count				
		20	50	100	150	200
	GPT-Neo(125M)	0/6	4/13	5/18	6/23	6/24
	GPT-Neo(1.3B)	3/16	4/24	6/30	7/32	7/32
ReAPR (No Retrieval)	GPT-Neo(2.7B)	6/23	6/28	6/34	8/41	11/47
	Phi-2(2.7B)	6/20	10/31	15/42	16/52	18/57
	DeepSeekCoder(6.7B)	29/75	36/90	39/98	43/109	44/115
	CodeLlama(7B)	22/54	30/71	36/87	42/100	**47/108**
	GPT-Neo(125M)	3/9	4/17	4/24	5/25	5/27
	GPT-Neo(1.3B)	5/14	7/19	8/26	9/30	10/31
ReAPR (Sparse Retrieval)	GPT-Neo(2.7B)	6/23	7/30	10/34	11/45	13/51
	Phi-2(2.7B)	13/27	15/40	17/48	20/55	21/61
	DeepSeekCoder(6.7B)	32/82	38/93	44/100	49/112	52/119
	CodeLlama(7B)	32/65	43/84	48/98	56/106	**59/113**
	GPT-Neo(125M)	2/5	3/14	3/21	3/22	4/25
	GPT-Neo(1.3B)	4/15	4/24	4/34	7/38	8/40
ReAPR (Dense Retrieval)	GPT-Neo(2.7B)	6/25	7/33	9/40	11/48	15/58
	Phi-2(2.7B)	9/32	19/51	21/59	24/61	26/70
	DeepSeekCoder(6.7B)	35/88	42/95	48/103	55/117	58/124
	CodeLlama(7B)	36/71	46/93	58/111	62/120	**68/130**

Repair results with sparse retrieval The row "ReAPR(Sparse Retrieval)" in Table 5 shows the results of the five generative models in the sparse retrieval setting where the retrievers use the BM25 algorithm. We first observe that similarly to the results in the no-retrieval setting, there is again a scaling effect on repair effectiveness. That is, increasing the size of the model also increases the number of plausible and correct patches generated. In addition, compared with the results under no retrieval setting, except for GPT-Neo(125M) model, the number of correct and plausible patches in the results of other models under sparse retrieval setting increased, and with the increase of model parameters, the number of correct and plausible patches increased more obviously. This indicates that the generation effect of RAG technology is more obvious for large models. We believe that this is because the larger the model is, the more it can use the retrieved knowledge to guide the model to repair the program and enhance the repair quality. However, to our surprise, the GPT-Neo(125M) model generated one fewer correct patches than the no-retrieval setting. We conjecture that this may be because the small number of model parameters makes it difficult to use the information of the retrieved bug-fix-pair to guide the model for program repair.

Repair results with dense retrieval The row "ReAPR(Dense Retrieval)" in Table 5 shows the results of the five generative models in the dense retrieval setting where the retriever uses the DPR algorithm. We first observe that similarly to the results in the no-retrieval and sparse-retrieval settings, there is again a scaling effect on repair effectiveness. That is, increasing the size of the model also increases the number of plausible and correct patches generated. At the same time, we also observe that the generation performance gap of the five generative models is further widened. For the Phi-2 model with high quality training data and the CodeLlama model which is suitable for code tasks and has a large number of parameters, the results after using dense retrieval, both correct patches and plausible patches, greatly outnumber the same models under no retrieval and sparse retrieval settings. However, it is interesting to note that for GPT-Neo(125M) and GPT-Neo(1.3B) models, their generated results are worse than for the model in the sparse retrieval setting. At the 200 patch setting, the correct patches are reduced by 1 and 2, respectively.

We attribute the result to the different algorithms used by the retriever. In the setting of sparse retrieval, the retrieval is based on the similarity of code words, so there are some same code words between the retrieved bug-fix-pair and the bug to be fixed, but their semantics may not be related to the bug to be fixed. Such bug-fix-pairs are more beneficial for GPT-Neo(125M) and GPT-Neo(1.3B) models, because small models have limited understanding of code semantics, and they are more sensitive and direct to capture relevant code vocabulary. For the large models with good performance, such as Phi-2 and CodeLlama, they have good code understanding ability, so the bug-fix-pair retrieved by dense retrieval has similar semantics with the bug to be fixed because it is retrieved according to semantic correlation. The large model can make good use of its repair information, and better guide the model to repair the program.

The performance of ReAPR on the new benchmark To more comprehensively evaluate the effectiveness of our approach, we also adopted a new benchmark–GitBug-Java. This benchmark consists of 199 bugs extracted from 55 well-known GitHub projects in 2023, making it the most recent Java benchmark available. We select all bugs confined to a single function (a total of 90) and generate 20 candidate patches for each bug. We then compare the results on GitBug-Java with those on Defects4J under the same setting of generating 20 candidate patches, as shown in Table 6. Our findings reveal that on GitBug-Java, small models such as the GPT-Neo series and Phi-2 barely repair any bugs, while medium-sized models like DeepSeekCoder and CodeLlama can only fix a very limited number of bugs (3 and 2 bugs, respectively). We attribute this to the fact that GitBug-Java collects bugs from open-source projects in the past two years, whereas the training corpora of models like CodeLlama predominantly consist of data collected before 2023. Consequently, the models have not encountered similar code snippets, leading to suboptimal code understanding. Furthermore, through manual inspection of GitBug-Java, we observe that many bug fixes depend on other functions within the same project. When only the buggy function is fed to the model, it lacks the necessary context and has not seen similar code fragments before, resulting in poor repair performance.

Furthermore, compared with Defects4J, we are surprised to find that on the GitBug-Java benchmark, neither sparse retrieval nor dense retrieval improved repair performance. In fact, using sparse retrieval with DeepSeekCoder resulted in one fewer bug being fixed compared to not using retrieval at all. We randomly inspect several retrieval samples and found that they differ significantly from the functions to be repaired. This discrepancy prevents the model from leveraging relevant repair experiences and may instead mislead the model, thereby hindering improvements in repair performance. This is because the retrieval corpus we have

Table 6 Results of generating 20 patches across two benchmark datasets

		Defects4j	
	No-Retrieval	Sparse-Retrieval	Dense-Retrieval
GPT-Neo(125M)	0	3	2
GPT-Neo(1.3B)	3	5	4
GPT-Neo(2.7B)	6	6	6
Phi-2(2.7B)	6	13	9
DeepSeekCoder(6.7B)	29	32	35
CodeLlama(7B)	22	32	36
		GitBug-Java	
	No-Retrieval	Sparse-Retrieval	Dense-Retrieval
GPT-Neo(125M)	0	0	0
GPT-Neo(1.3B)	1	0	0
GPT-Neo(2.7B)	0	0	1
Phi-2(2.7B)	0	0	0
DeepSeekCoder(6.7B)	3	2	3
CodeLlama(7B)	2	2	3

constructed is built from open-source projects on GitHub prior to 2023, and therefore contains no related data for the bugs in GitBug-Java. This demonstrates that the key factor affecting RAG performance lies in the quality of the retrieval corpus; if no relevant content exists in the corpus, RAG loses its effectiveness and may even harm the model's performance.

Answer to RQ1: Our proposed ReAPR achieves repair performance comparable to SOTA APR methods even without retrieval. With retrieval enabled, ReAPR's repair performance consistently surpasses current SOTA methods. For ReAPR, as the model parameters increases, the use of retrieval significantly enhances the model's repair performance, with dense retrieval proving to be more effective than sparse retrieval. Conversely, for smaller models, the use of retrieval does not improve repair performance and may even hinder it. Moreover, the results on the new benchmark indicate that RAG heavily depends on the quality of the retrieval corpus. For bugs that have no related entries in the corpus, even high-performing models like CodeLlama cannot leverage retrieval to improve repair performance; instead, retrieval may degrade the model's repair effectiveness.

5.2 RQ2: Patch accuracy correlation with retrieval settings

We evaluate the correct rate of patches generated by each LLMs (i.e., correct patches/plausible patches). The result is a little different from what we expected. We believe that regardless of the retrieval setting, the larger the model parameter is, the lower the error rate of the generated patches will be (i.e., the proportion of correct patches to plausible patches will be large). However, we examine the number of correct and plausible patches corresponding to the six generative models under three different retrieval settings when the number of generated patches is 200, as shown in Fig. 6. In general, it is indeed consistent with our expected results. No matter which retrieval setting, the correct rate of generated patches is

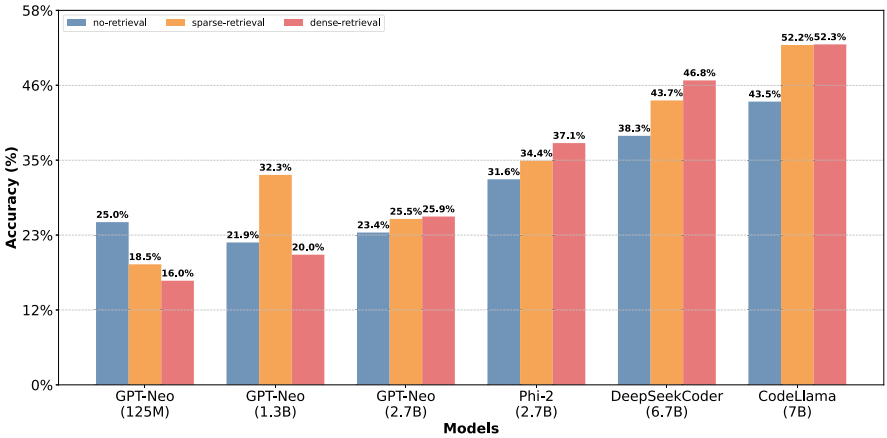
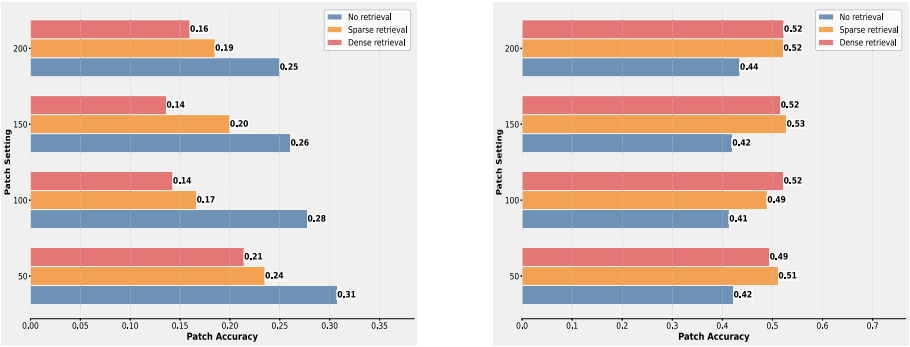


Fig. 6 The patch accuracy under the 200 patch setting

gradually improved with the increase of model parameters. In particular, the compilation correct rate of CodeLlama model under dense retrieval setting reaches 52.3%. This shows that with the increase of the model, the performance of the model is continuously enhanced, not only the number of correct and trusted patches is increased, but also the patch accuracy is also improved in general, which proves that the patch quality generated by the large model is better than that of the small model.

However, we also observe that the patch accuracy of the model is not strictly increasing in the no-retrieval and sparse retrieval settings. For the GPT family of models, GPT-Neo (125M) has a patch accuracy of 25.0% in the no-retrieval setting, while GPT-Neo (1.3T) with a larger number of parameters has a patch accuracy of only 21.9% in the no-retrieval setting. In addition, GPT-Neo(1.3B) achieves 32.3% patch accuracy in the sparse retrieval setting, while GPT-Neo(2.7B) with a larger number of parameters only achieves 25.5% patch accuracy in the sparse retrieval setting. We believe that the reason for this result is that the GPT family of models is mainly trained on text, and code is only a small part of the training data, so the models do not perform very well on code-related tasks such as APR. However, the CodeLlama model trained specifically on code shows excellent ability, which further verifies our conjecture. Interestingly, for dense retrieval, the patch accuracy of all the generated models increases strictly monotonically with the increase of model parameters, which preliminarily proves that retrieval Settings have an impact on the patch correctness of the model.

We separately take out the two more typical models, GPT-Neo(125M) and CodeLlama(7B), for a more detailed analysis. We evaluate the patch correctness rate under three different retrieval settings and five different patch generation settings, and the results are shown in Fig. 7. Figure 7(a) shows the patch accuracy of the GPT-Neo(125M) model under these Settings, and we observe that the patch accuracy of no retrieval is greater than that of sparse retrieval than that of dense retrieval under each other set of settings. It can be seen that for GPT-Neo(125M) model, the use of retrieval does not improve the patch generation quality, but harms the patch generation quality. Moreover, compared with sparse retrieval, dense retrieval has more damage to patch generation performance of GPT-Neo(125M) model. However, Figure 7(b) shows exactly the opposite result from Fig. 7(a). For the CodeLlama(7B) model, the patch accuracy of dense retrieval and sparse retrieval is higher than that of no



(a) Patch accuracy for GPT-Neo(125M) (b) Patch accuracy for CodeLlama(7B)

Fig. 7 Patch accuracy of the two models under different patch Settings

retrieval. It can be seen that for the CodeLlama(7B) model, the use of retrieval will improve the patch accuracy, generate higher quality patches, and reduce the illusion of the model.

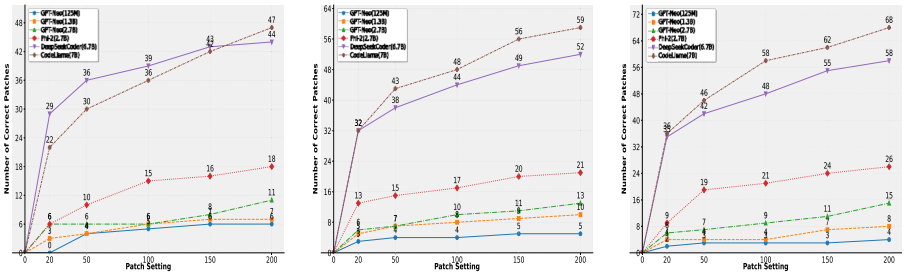
Answer to RQ2: For LLMs like CodeLlama(7B), the use of retrieval can improve patch accuracy; however, for smaller models such as GPT-Neo(125M), the use of retrieval may actually decrease patch accuracy.

5.3 RQ3: Correct patches correlation with generation settings

For the APR task, patch generation time is very expensive, especially for the large model such as CodeLlama(7B). We measure the time each model takes to generate a single patch on the GitBug-Java dataset, with the results shown in Table 7. We find that for small models, the patch generation speed is relatively fast and the differences among them are generally minor. However, for the two medium-sized models, DeepSeekCoder and CodeLlama, the time required to generate a single patch is more than ten times longer than that of the small models. We attribute this discrepancy partly to the difference in model sizes, as DeepSeekCoder and CodeLlama involve significantly higher computational costs compared to small models. Additionally, small models have input token length limitations–for example, the Phi-2 model supports only up to 2048 tokens–restricting their ability to process longer function bodies. On the other hand, by examining the patches generated, we observed that even when the buggy code spans dozens of lines, the patches produced by small models tend to be relatively short, sometimes only 2 to 3 lines. In contrast, medium-sized models like CodeLlama generate patches that closely match the length of the buggy code. Because these models produce more tokens, the time required for patch generation increases correspondingly.

Table 7 The time required for the model to generate each patch

	GPT-Neo(125M)	GPT-Neo(1.3B)	GPT-Neo(2.7B)
Patch Generation Time	2.8s	4.5s	5.9s
	Phi-2(2.7B)	DeepSeekCoder(6.7B)	CodeLlama(7B)
Patch Generation Time	6.3s	75.5s	85.2s



(a) The number of correct patches in no retrieval setting

(b) The number of correct patches in sparse retrieval setting

(c) The number of correct patches in dense retrieval setting

Fig. 8 Relationship between the number of correct patches and the generated patch Settings under three different retrieval Settings

In order to trade off the generation patch quality and generation efficiency, we let each group of LLMs generate 20, 50, 100, 150, 200 patches to find the best setting of the number of generated patches, as shown in Fig. 8. Figure 8(a), (b) and (c) detail the variation trend of the number of correct patches as the number of generated patches increases under different retrieval settings, respectively. We find that for the model with small parameters, the growth rate of the number of correct patches gradually slows down after the patch number is set to 50. However, this situation is not obvious for the model with a large number of parameters, which indicates that the correctness of patches generated by the small model is very limited, while the large model can generate a large number of high-quality correct patches with good semantic understanding. Therefore, for small models, it is reasonable to set the optimal number of generated patches as 50 under this experimental result, while for large models, such as CodeLlama(7B), there is no appropriate value as the optimal number of generated patches under this experimental result, and the specific situation should be viewed according to specific requirements.

Answer to RQ3: Balancing the accuracy of generated patches and generation efficiency requires a specific analysis based on the characteristics of the model. For smaller models, setting the optimal number of generated patches to 50 is reasonable; however, for larger models, there is no suitable value, and it should be considered in relation to the specific focus.

5.4 RQ4: Comparison of different numbers of bug-fix-pairs

In addition, we also study the effect of feeding different numbers of bug-fix-pairs to the model, on the generated plausible patches and correct patches. Following (Cuconasu et al., 2024), we place the bug-fix-pair with the highest semantic similarity to the bug to be fixed closest to the bug to be fixed, and the remaining bug-fix-pairs are located further and further away from the bug to be fixed as their similarity scores decrease. Due to the limited memory of GPUs, here we only study the impact of three bug-fix-pairs on the patching performance, and the results are shown in Table 8. We find that the number of plausible patches and correct patches decreases for both two and three bug-fix-pairs compared to the number of results for one bug-fix-pair. We think that this is because the model is affected by those bug-fix-pairs that are irrelevant to the bug to be fixed, and previous studies (Cuconasu et al., 2024) have shown that irrelevant search results can harm the inference results of the model in RAG

Table 8 CodeLlama(7B) results for different numbers of bug-fix-pairs in the dense retrieval setting

The number of bug-fix-pairs	20	50	100	150	200
1 pair	36/71	46/93	58/111	62/120	68/130
2 pairs	31/58	38/78	49/104	55/115	57/122
3 pairs	37/67	45/89	55/100	58/111	59/116

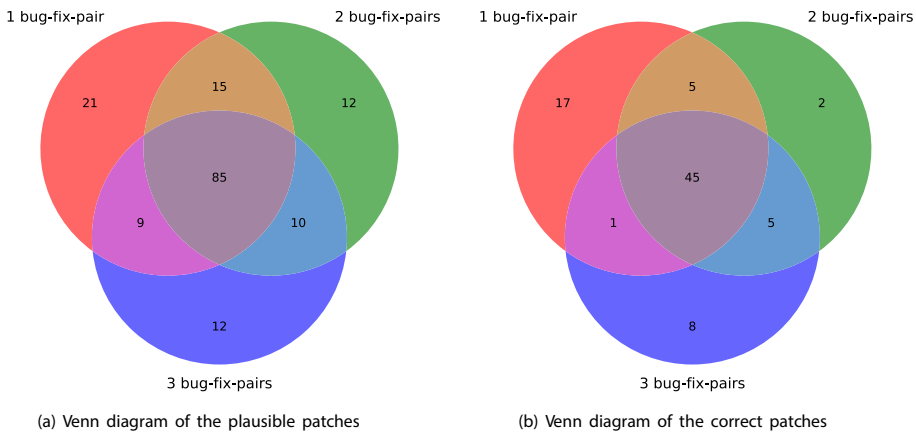
systems. Moreover, with the increase of bug-fix-pair, the prompt will become very long, and the model has limited processing ability for long texts, and it is difficult to capture correct repair experience, which will make the repair effect of the model worse.

Figure 9 shows the unique fix Venn diagram generated by feeding CodeLlama (7B) different numbers of bug-fix-pairs in the setting of dense retrieval. Figure 9a and b show the unique fixes of plausible patches and correct patches, respectively. We observe that although these three sets of experimental results show that they can generate a large number of the same patch fixes, no matter which set of experimental results, they can generate some unique fixes. For example, the results of 1 bug-fix-pair in Fig. 9b show that it can generate 17 unique fixes, while 2 and 3 bug-fix-pairs can generate 2 and 8 unique fixes, respectively. Our results show that the number of bug-fix-pairs is also one of the factors affecting the repair performance.

Answer to RQ4: Although increasing the number of bug-fix pairs may yield some unique repairs, it does not enhance the model's repair performance; on the contrary, it may hinder the model's ability to fix bugs.

5.5 RQ5: Comparison with PEFT approaches

Finally, we compared the performance of RAG and Parameter-Efficient Fine-Tuning (PEFT) on automatic program repair, as well as their combined effect, as shown in Table 9. In this study, we adopted the CodeLlama model fine-tuned by Silva et al. (2023), which we refer to as CodeLlama-LoRA. CodeLlama-LoRA was fine-tuned on the MegaDiff dataset, which is



(a) Venn diagram of the plausible patches (b) Venn diagram of the correct patches

Fig. 9 Venn diagram of the number of plausible and correct patches under the 200 number of generated patches setting

Table 9 RAG vs PEFT across two benchmark datasets

		Defects4j	
	No-Retrieval	Sparse-Retrieval	Dense-Retrieval
CodeLlama(7B)	22	32	36
CodeLlama-Lora(7B)	41	51	54
		GitBug-Java	
	No-Retrieval	Sparse-Retrieval	Dense-Retrieval
CodeLlama(7B)	2	2	3
CodeLlama-Lora(7B)	4	3	4

also included in the retrieval corpus used by our RAG approach. For each bug, we generated 20 candidate patches. The comparison reveals that both RAG and PEFT improve model performance on Defects4J, with PEFT repairing 5 more bugs than RAG alone. Moreover, combining PEFT and RAG yields synergistic effects, successfully repairing 54 bugs–18 more than RAG alone and 13 more than PEFT alone. However, on the GitBug-Java dataset, neither RAG nor PEFT approaches significantly enhance repair capabilities. This stems from their inherent reliance on learning repair patterns from training or retrieved corpora (e.g., MegaDiff, which aggregates pre-2023 GitHub projects). Crucially, these corpora lack repair cases analogous to the target bugs in GitBug-Java. This underscores the pivotal role of training/retrieval corpora relevance in model performance.

Answer to RQ5: In APR tasks, both RAG and PEFT essentially aim to incorporate more repair knowledge into the model, derived from retrieval corpora and training corpora. If these corpora contain bug-fix pairs relevant to the bugs to be repaired, both RAG and PEFT can improve repair performance. Moreover, their combination yields even more significant improvements. However, if neither the retrieval nor the training corpora include related bug-fix pairs, neither RAG nor PEFT can effectively enhance repair performance, and combining them also fails to bring improvements.

5.6 Case study

Retrieval *v.s.* no retrieval To visualize the effectiveness of our methods, we show some of the unique fixes they produce. As shown in Fig. 10, we find that this function cannot be repaired in the no-retrieval setting using the same CodeLlama(7B) model, while after using the retrieval method (both sparse and dense retrieval can be used), the function is successfully repaired. The bug is caused by a missing **`Double.isNaN(value)`** condition in the **`if`** statement when the **`lenient`** flag is set to `false` for the function **`value`**. Since the function body does not mention this condition, this condition is lenient, so even the well-performing

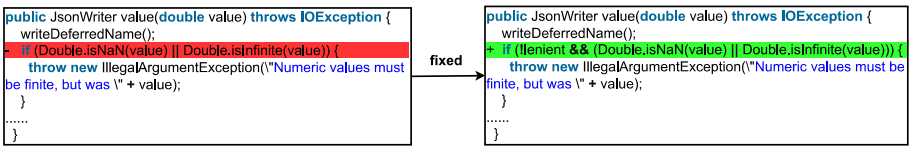


Fig. 10 Bugs and fixes for Gson-15

```
public JsonWriter writeNumberValue(Number value) throws IOException {
    if (value == null) {
        return nullValue();
    }
    String string = value.toString();
-   if (string.equals(\"String_Node_Str\") || string.equals(\"String_Node_Str\")
|| string.equals(\"String_Node_Str\")) {
        throw new IllegalArgumentException(\"String_Node_Str\" + value);
    }
......
}
```

(a) Related buggy code for Gson-15

```
public JsonWriter writeNumberValue(Number value) throws IOException {
    if (value == null) {
        return nullValue();
    }
    String string = value.toString();
+   if (lenient && (string.equals(\"String_Node_Str\") || string.equals(\"String_
Node_Str\") || string.equals(\"String_Node_Str\"))) {
        throw new IllegalArgumentException(\"String_Node_Str\" + value);
    }
......
}
```

(b) Related fixed code for Gson-15

Fig. 11 The bug-fix-pairs associated with Gson-15 retrieved using the retrievers

CodeLlama (7B) model will not fix the bug in a general APR flow. The function is derived from a real-world open source project, showing that this kind of error is very easy to occur in daily programming, and once it occurs, it is difficult to fix according to normal methods.

At this point, the effectiveness of our method is demonstrated, because the function can be repaired correctly regardless of whether sparse or dense retrieval is used. To explore the reason why this function is fixed, we retrieve the corresponding bug-fix-pair of this function from the retrieval database using sparse and dense retrieval, respectively, as shown in Fig. 11. Figure 11(a) shows the corresponding bugged function body of this function, and Fig. 11(b) shows the corresponding fixed function body. We observe that although the specific implementation details of the bug-fix-pair are slightly different from the original Gson-15 function to be fixed, the CodeLlama(7B) model can learn the fixing experience of the bug-fix-pair very well, which is very helpful for fixing the Gson-15 function. This is because the bug in this bug-fix-pair is also because **`if (string.equals(\"String_Node_Str\) || string.equals(\"String_Node_Str\) || string.equals(\"String_Node_Str\))`** statement missing **`!lenient`**. Therefore, the bug-fix-pair is a good guide for the model to fix the corresponding function.

Dense retrieval *v.s.* sparse retrieval In addition, we study the bugs that only dense retrieval can fix and their corresponding bug-fix-pair retrieved. Likewise, in order to explore why sparse retrieval cannot fix the bug, we also take out the corresponding bug-fix-pair retrieved by sparse retrieval algorithm and compare with the densely retrieved bug-fix-pair. As shown in Fig. 12, the bug in the readChar() function occurs in the if statement **`if (current == '\n')`**, which is fixed as **`if (current == '\r' || (current == '\n' && lastChar != '\r'))`**. We can see that to fix this bug, we need to add an extra condition to the if statement, and as explained earlier, there is no reference to the \r character in the body of the function, so fixing this bug is quite difficult.

As shown in Fig. 13, Figure 13(a) shows the bug-fix-pair retrieved by the DPR algorithm. We can see that this `readChar()` function has roughly the same semantics as the `read()` function to be fixed. The bugs in `readChar()` and their fixes are good lessons for the

```
public int read() throws IOException {
    int current = super.read();
-   if (current == '\\n') {
        lineCounter++;
    }
    lastChar = current;
    return lastChar;
}
```

fixed →

```
public int read() throws IOException {
    int current = super.read();
+   if (current == '\\r' || (current == '\\n' && lastChar != '\\r')) {
        lineCounter++;
    }
    lastChar = current;
    return lastChar;
}
```

Fig. 12 Bugs and fixes for Csv-1

```
private int readChar() throws java.io.IOException {
    int c = reader.read();
-   if ((c == '\r') || ((c == '\n') && ((lastc) != '\r'))) {
+   if (((lastc) == '\n') || (((lastc) == '\r') && (c != '\n'))) {
        pos = 1;
        (line)++;
    }else {
        (pos)++;
    }
    lastc = c;
    return c;
}
```

(a) Related bug-fix-pair for Csv-1 (DPR)

```
private static Document load(File file) throws Exception {
    final long fileLength = file.length();
    SAXReader saxReader = new SAXReader();
-   Document document = saxReader.read(new FileReader(file) {
+   InputStreamReader reader = new InputStreamReader(new FileInputStre
am(file), \"String_Node_Str\") {
        public int read(char[] chars) throws IOException {
            int n = super.read(chars);
......
        public int read(char[] chars, int start, int length) throws IOException {
            int n = super.read(chars, start, length);
......
```

(b) Related bug-fix-pair for Csv-1 (BM25)

Fig. 13 The bug-fix-pairs associated with Csv-1 retrieved using the retrievers

`read()` function to be fixed. However, the function semantics of the bug-fix-pair retrieved by the BM25 algorithm shown in Fig. 13(b) is quite different from that of the function to be fixed. Therefore, the repair experience provided by the bug-fix-pair is not helpful for the function `read()` to be fixed, which makes the bug unable to be fixed in the sparse retrieval setting. So why is this the case for the bug-fix-pair retrieved using the BM25 algorithm? We know that the BM25 algorithm calculates the similarity score based on the word frequency. In the `load()` function, we observe that it contains a lot of the word – read, resulting in a high similarity score, but with a completely different semantics than the function to be fixed. In addition, we also observed the bug-fix-pairs retrieved by the BM25 algorithm for functions in some other projects, and found that they were similar to the results of the Csv-1 project, containing a large number of the same words, but with great semantic differences. This also proves the superiority of the DPR algorithm over the BM25 algorithm.

Discussion on data leakage Data leakage is a critical factor that can affect the validity of a method. To investigate whether data leakage exists in our work, we took dense retrieval as an example and manually examined every bug-fix pair retrieved by the dense retriever on the Defects4J benchmark. We then compared each retrieved bug-fix pair with the corresponding bug-fix pair in Defects4J. Our comparison revealed that none of the retrieved examples were exactly the same as the bugs in the Defects4J benchmark. This is because, during the construction of the retrieval corpus, we had already taken this issue into consideration and excluded any bug-fix pairs that were identical to those in Defects4J.

Furthermore, we observed that among the successfully repaired cases after retrieval, some instances are similar to the previously mentioned cases such as Gson-15 and Csv-1. This aligns with our expectations: RAG helps the large language model fix more bugs precisely because it provides the model with useful repair experiences to draw upon. This also demonstrates the effectiveness of our method.

6 Related work

In this section, we investigate the related work from the perspectives of LLMs for Code, APR, and RAG for Code-related tasks.

6.1 LLMs for code

To better adapt LLMs to code-related tasks, many works have extended NLP pre-training methods to source code. CuBERT (Kanade et al., 2020) and CodeBERT (Feng et al., 2020) were the two pioneer models. CuBERT employed BERT's powerful masked language modeling objective to derive generic code-specific representation, and CodeBERT further added a replaced token detection (Clark et al., 2020) task to learn NL-PL cross-modal representation. GraphCodeBERT (Guo et al., 2020) incorporated the data flow extracted from the code structure into CodeBERT, while Roziére et al. (2021) proposed a deobfuscation objective to leverage the structural aspect of code. These models only focused on training a better code-specific encoder. Wang et al. (2021) explored encoder-decoder models based on T5 for programming language pre-training and support a more comprehensive set of tasks. Apart from these, PLBART (Ahmad et al., 2021) based on another encoder-decoder model BART can also support both understanding and generation tasks. Recently, Meta introduced CodeLlama (Roziere et al., 2023), a model that fine-tunes the recent general language model Llama 2 on code data and provides state-of-the-art performance among open models.

6.2 APR

Existing approaches to APR can be categorized as two groups: template-based (Just et al., 2014; Li et al., 2022; Harer et al., 2018; Li et al., 2020; Le et al., 2019) and deep-learning-based (Tan & Roychoudhury, 2015; Chen et al., 2017; Weimer et al., 2009; Long & Rinard, 2015). In 2019, Koyuncu et al. (2019) developed iFixR, an APR tool driven by bug reports. iFixR builds on kPAR (Liu et al., 2019) (a template-based APR tool) and uses its repair templates. Similar to iFixR (Koyuncu et al., 2019), Liu et al. (2019a) also summarized 35 repair templates from previous template-based works and proposed an APR tool TBar. DL technologies continued to drive APR in 2020. Li et al. (2020) proposed DLFix, which used trees (AST) to record code structure. This allowed DLFix to better take context into account and generate higher quality fixes. Furthermore, Lutellier et al. (2020) proposed CoCoNuT, which uses CNNS to extract hierarchical features, which helps to capture rich (multi-level) semantic information. Recently, the emergence of LLMs has given new directions to the APR field. Mashhadi and Hemmati (2021) first used CodeBERT fine-tuning to solve single-line bug repair problems. Later, Huang et al. (2022) investigated the repair effectiveness of using CodeBERT and GraphCodeBERT based on fine-tuning. Recently, Xia and Zhang (2022) introduced cloze tasks into the APR domain, which uses LLMCs to predict the correct code for defect locations with context. They proposed the APR tool AlphaRepair based on CodeBERT and zero-shot learning, which provides a new direction towards APR. In addition, Xia et al. (2023) proposed another novel APR tool, FitRepair, based on CodeT5, which combines the plastic surgery hypothesis with LLMCs to provide additional repair ingredients and thus enhance the repair capability of LLMCs. With the recent popularity of ChatGPT (Wu et al., 2023), Xia and Zhang (2023) proposed ChatRepair, a conversational APR tool that provides a new workflow using LLMs by continuously learning knowledge and receiving feedback to enhance repair capability. Wang et al. introduced RAP-Gen (Wang et al., 2023), which employs CodeT5 (an encoder–decoder model) with retrieval happening in the encoder and generation in the decoder. ThinkRepair (Yin et al., 2024) by Xin Yin et al. focuses on retrieving bug–CoT–fix triples generated via LLM prompting and validated through test cases. Xia et al. enhanced generation quality in Repilot (Wei et al., 2023) by integrating a rule-based post-processor from Eclipse JDT that masks invalid tokens. Efficiency is the main focus

of FLAMES (Le-Cong et al., 2024), developed by LE-CONG et al., which utilizes greedy decoding alongside a test-guided patch refinement loop. Meanwhile, Silva et al. explored the effectiveness of PEFT for automated program repair by applying it on CodeLlama-7B in their RepairLlama (Silva et al., 2023) approach.

6.3 RAG for code-related tasks

Many code intelligence tasks benefit from information retrieval (Xia et al., 2017). RAG is a general paradigm for information retrieval which enhances LLMs by including relevant information retrieved from external databases into the input. RAG typically consists of three phases: indexing, retrieval, and generation. First, the indexing phase constructs external databases and their retrieval index from external data sources; Second, given a user query, the retrieval system then utilizes these index to fetch the relevant document chunks as context. Third, the retrieved context is then integrated into the input prompt for LLMs, and LLMs then generate the final output based on the augmented inputs.

RAG has been widely used in various fields. For example, RAG has been specialized for software engineering tasks and performs well in many code intelligence tasks. A common scenario for information retrieval in code domain is code search with natural language description as a query (Arwan et al., 2015; Gu et al., 2018; Cambronero et al., 2019). As for other code intelligence tasks, Hayati et al. (2018) proposed an action subtree retrieval method called ReCode for generating general-purpose code. Hashimoto et al. (2018) proposed a retrieve-and-edit framework for code auto completion and code generation. Luan et al. (2019) proposed Aroma, which utilizes code-to-code structural search and intersecting candidate code snippets to recommend relevant code given another code snippet as a query. Both Wei et al. (2020) and Li et al. (2021) leveraged the retrieve-and-edit/refine framework to improve model's performance in code summarization. Parvez et al. (2021) proposed REDCODER, using a dense retriever trained on paired NL-code pairs to retrieve relevant comments or codes as a supplement for code summarization or code generation tasks. Lu et al. (2022) proposed ReACC to construct a hybrid retriever to retrieve relevant codes for the code completion task by combining sparse and dense retrievers. While in most circumstances utilizing dense retrievers, natural language comments are treated as queries to retrieve code, however, in the APR task, we focus on using code as queries, particularly bug-containing code as queries.

7 Threats to validity

There are three primary threats to the validity for our work.

Single programming language While our extensive experiments focused on repairing complete functions in Java, we did not extend our approach to other programming languages such as C or Python. Expanding this work would require the collection of repair instances in these languages to build a more comprehensive retrieval dataset, which we plan to explore in future research.

Limited model size Finally, our experiments demonstrate that as model size increases, the repair performance using RAG becomes more pronounced. However, due to limited computational resources, our largest model was capped at 7B parameters, and we were unable to test larger models, such as those with 13B, 34B, or 70B parameters. Despite this

limitation, we believe our findings are representative across a broad range of conditions. Future work, supported by more advanced computational resources, could further enhance and build upon our study.

8 Conclusion

In this paper, we have proposed ReAPR, a retrieval-augmented APR framework that provides additional “external knowledge" to the APR task to better guide LLMs for program repair by retrieving bug-fix-pairs that are semantically and lexically similar to the bug to be fixed from the retrieval database. We construct a sparse retriever using the BM25 algorithm provided by the ElasticSearch search engine, while also training and fine-tuning GraphCodeBERT as a dense retriever to reduce the illusion of LLMs and generate higher quality fixes by retrieving related bug-fix-pairs. We also compare the effectiveness of RAG and PEFT techniques, as well as their combination, for automated program repair. We evaluate ReAPR on the widely used Defects4j 2.0 benchmark and the latest GitBug-Java benchmark datasets. The results show that when the training corpus and retrieval corpus contain bug-fix pairs related to the bugs to be fixed, leveraging the retrieved information can significantly improve repair performance. Moreover, combining RAG with PEFT can mutually enhance and jointly improve repair capability. In particular, CodeLlama (7B) fixed 68 and 59 bugs in Defects4j 2.0 under dense and sparse retrieval settings, respectively, which is 18 and 9 more bugs than the best baseline method, verifying the effectiveness of our approach. However, we also find that if the training and retrieval corpora do not contain bug-fix pairs related to the bugs to be fixed, neither RAG nor PEFT can improve the model’s repair performance, rendering both ineffective in such cases.

Author Contributions Zixin Liu: Conceptualization, Methodology, Software, Data curation, Investigation, Writing – original draft. Xiaozhi Du: Resources, Supervision. Hairui Liu: Data curation.

Funding The work was supported in part by the Chinese National Natural Science Foundation (grant number 12275208).

Data Availability The retrieval database supporting the results of this study is openly available in the huggingface repository. The dataset can be accessed at the following URL: https://huggingface.co/datasets/zxliu/ReAPR-Automatic-Program-Repair-via-Retrieval-Augmented-Large-Language-Models. And we have open-sourced our code on GitHub. The repository is available at: https://github.com/ZXLiu/ReAPR.

Declarations

Competing interests The authors declare no competing interests.

References

Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Unified pre-training for program understanding and generation. arXiv:2103.06333.

Arwan, A., Rochimah, S., Akbar, R.J.: Source code retrieval on stackoverflow using lda. In: *2015 3rd International Conference on Information and Communication Technology (ICoICT)*, pp. 295–299 (2015). IEEE

Black, S., Gao, L., Wang, P., Leahy, C., & Biderman, S. (2021). Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow. *If you use this software, please cite it using these metadata*. *58*(2).

Brown, T. B. (2020). Language models are few-shot learners. ArXiv:2005.14165.

Cambronero, J., Li, H., Kim, S., Sen, K., & Chandra, S. (2019). When deep learning met code search. In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 964–974.

Chen, Z., & Monperrus, M. (2018). The codrep machine learning on source code competition. arXiv preprint arXiv:1807.03200.

Chen, L., Pei, Y., & Furia, C. A. (2017). Contract-based program repair without the contracts. In: *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 637–647. IEEE.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). Evaluating large language models trained on code. arXiv:2107.03374

Clark, K., Luong, M.-T., Le, Q. V., & Manning, C. D. (2020). Electra: Pre-training text encoders as discriminators rather than generators. arXiv:2003.10555.

Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonellotto, N., Silvestri, F. (2024). The power of noise: Redefining retrieval for rag systems. In: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 719–729.

Devlin, J. (2018). Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv:1810.04805.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155. (2020)

Fried, D., Aghajanyan, A., Lin, J., Wang, S., Wallace, E., Shi, F., Zhong, R., Yih, W.-t., Zettlemoyer, L., & Lewis, M. (2022). Incoder: A generative model for code infilling and synthesis. arXiv:2204.05999.

Gu, X., Zhang, H., & Kim, S. (2018). Deep code search. In: *Proceedings of the 40th International Conference on Software Engineering*, pp. 933–944.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al. (2020). Graphcodebert: Pre-training code representations with data flow. arXiv:2009.08366.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. (2024). Deepseek-coder: When the large language model meets programming–the rise of code intelligence. arXiv:2401.14196.

Harer, J., Ozdemir, O., Lazovich, T., Reale, C., Russell, R., Kim, L., et al. (2018). Learning to repair software vulnerabilities with generative adversarial networks. *Advances in Neural Information Processing Systems*. *31*.

Hashimoto, T. B., Guu, K., Oren, Y., & Liang, P. S. (2018). A retrieve-and-edit framework for predicting structured outputs. Advances in Neural Information Processing Systems. *31*.

Hayati, S.A., Olivier, R., Avvaru, P., Yin, P., Tomasic, A., & Neubig, G. (2018). Retrieval-based neural code generation. arXiv:1808.10025.

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2019). The curious case of neural text degeneration. arXiv:1904.09751.

Huang, K., Yang, S., Sun, H., Sun, C., Li, X., & Zhang, Y. (2022). Repairing security vulnerabilities using pre-trained programming language models. In: *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, pp. 111–116. IEEE.

Javaheripi, M., Bubeck, S., Abdin, M., Aneja, J., Bubeck, S., Mendes, C.C.T., Chen, W., Del Giorno, A., Eldan, R., Gopi, S., et al. (2023). Phi-2: The surprising power of small language models. *Microsoft Research Blog*.

Jiang, N., Liu, K., Lutellier, T., & Tan, L. (2023). Impact of code language models on automated program repair. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1430–1442. IEEE

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., Yang, Y., Callan, J., & Neubig, G. (2023). Active retrieval augmented generation. arXiv preprint arXiv:2305.06983.

Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4j: A database of existing faults to enable controlled testing studies for java programs. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, pp. 437–440.

Kanade, A., Maniatis, P., Balakrishnan, G., & Shi, K. (2020). Learning and evaluating contextual embedding of source code. In: *International Conference on Machine Learning*, pp. 5110–5121. PMLR.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, & W.-t. (2020). Dense passage retrieval for open-domain question answering. arXiv:2004.04906.

Kolak, S. D., Martins, R., Le Goues, C., & Hellendoorn, V. J. (2022). Patch generation with language models: Feasibility and scaling behavior. In: *Deep Learning for Code Workshop*.

Koyuncu, A., Liu, K., Bissyandé, T.F., Kim, D., Monperrus, M., Klein, J., & Le Traon, Y. (2019). ifixr: Bug report driven program repair. In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 314–325.

Lachaux, M.-A., Roziere, B., Szafraniec, M., & Lample, G. (2021). Dobf: A deobfuscation pre-training objective for programming languages. *Advances in Neural Information Processing Systems, 34*, 14967–14979.

Le, X.-B. D., Bao, L., Lo, D., Xia, X., Li, S., & Pasareanu, C. (2019). On reliability of patch correctness assessment. In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pp. 524–535. IEEE.

Le-Cong, T., Le, B., & Murray, T. (2024). Semantic-guided search for efficient program repair with large language models. arXiv preprint arXiv:2410.16655.

Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2019). Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. arXiv:1910.13461.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-T., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474.

Li, J.A., Li, Y., Li, G., Hu, X., Xia, X., & Jin, Z. (2021). Editsum: A retrieve-and-edit framework for source code summarization. In: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 155–166. IEEE.

Li, H., Su, Y., Cai, D., Wang, Y., & Liu, L. (2022). A survey on retrieval-augmented text generation. arXiv:2202.01110.

Li, Y., Wang, S., & Nguyen, T. N. (2020). Dlfix: Context-based code transformation learning for automated program repair. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pp. 602–614.

Li, Y., Wang, S., & Nguyen, T.N. (2020). Dlfix: Context-based code transformation learning for automated program repair. In: *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pp. 602–614.

Li, Y., Wang, S., & Nguyen, T. N. (2022). Dear: A novel deep learning-based approach for automated program repair. In: *Proceedings of the 44th International Conference on Software Engineering*, pp. 511–523.

Liu, Y. (2019). Fine-tune bert for extractive summarization. arXiv:1903.10318.

Liu, K., Koyuncu, A., Bissyandé, T. F., Kim, D., Klein, J., & Le Traon, Y. (2019). You cannot fix what you cannot find! an investigation of fault localization bias in benchmarking automated program repair systems. In: *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, pp. 102–113. IEEE.

Liu, K., Koyuncu, A., Kim, D., & Bissyandé, T. F. (2019). Avatar: Fixing semantic bugs with fix patterns of static analysis violations. In: *2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pp. 1–12. IEEE.

Liu, K., Koyuncu, A., Kim, D., & Bissyandé, T. F. (2019). Tbar: Revisiting template-based automated program repair. In: *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 31–42.

Long, F., & Rinard, M. (2015). Staged program repair with condition synthesis. In: *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pp. 166–178.

Lu, S., Duan, N., Han, H., Guo, D., Hwang, S.-W., & Svyatkovskiy, A. (2022). Reacc: A retrieval-augmented code completion framework. arXiv:2203.07722.

Luan, S., Yang, D., Barnaby, C., Sen, K., & Chandra, S. (2019). Aroma: Code recommendation via structural code search. *Proceedings of the ACM on Programming Languages*. *3*(OOPSLA), 1–28.

Lutellier, T., Pham, H.V., Pang, L., Li, Y., Wei, M., & Tan, L. (2020). Coconut: combining context-aware neural translation models using ensemble for program repair. In: *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 101–114.

Mashhadi, E., Hemmati, H. (2021). Applying codebert for automated program repair of java simple bugs. In: *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pp. 505–509. IEEE.

Matteson, S. (2018). Report: Software failure caused $1.7 trillion in financial losses in 2017. TechRepublic.

Monperrus, M., Martinez, M., Ye, H., Madeiral, F., Durieux, T., & Yu, Z. (2021). Megadiff: A dataset of 600k java source code changes categorized by diff size. arXiv:2108.04631.

Oord, A. v. d., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. arXiv:1807.03748.

Parvez, M. R., Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Retrieval augmented code generation and summarization. arXiv:2108.11601.

Pepe, F., Nardone, V., Mastropaolo, A., Bavota, G., Canfora, G., & Di Penta, M. (2024). How do hugging face models document datasets, bias, and licenses? an empirical study. In: *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension*, pp. 370–381.

Prenner, J. A., Babii, H., & Robbes, R. (2022). Can openai's codex fix bugs? an evaluation on quixbugs. In: *Proceedings of the Third International Workshop on Automated Program Repair*, pp. 69–75.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., & Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research, 21*(140), 1–67.

Richards, E. (1990). Software's dangerous aspect. The Washington Post 25.

Robertson, S., Zaragoza, H., *et al.*: The probabilistic relevance framework: Bm25 and beyond. Foundations and Trends® in Information Retrieval. **3**(4), 333–389 (2009)

Roziere, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. (2023). Code llama: Open foundation models for code. arXiv:2308.12950.

Shazeer, N. (2019). Fast transformer decoding: One write-head is all you need. arXiv:1911.02150.

Silva, A., Fang, S., Monperrus, M. (2023). Repairllama: Efficient representations and fine-tuned adapters for program repair. arXiv:2312.15698.

Silva, A., Saavedra, N., & Monperrus, M. (2024). Gitbug-java: A reproducible benchmark of recent java bugs. In: *Proceedings of the 21st International Conference on Mining Software Repositories*, pp. 118–122.

Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*. *27*.

Tan, S. H., & Roychoudhury, A. (2015). relifix: Automated repair of software regressions. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1, pp. 471–482. IEEE.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288.

Tufano, M., Watson, C., Bavota, G., Penta, M. D., White, M., & Poshyvanyk, D. (2019). An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM), 28*(4), 1–29.

Tufano, M., Watson, C., Bavota, G., Penta, M. D., White, M., & Poshyvanyk, D. (2019). An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM), 28*(4), 1–29.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. Advances in neural information processing systems. *30*.

Wang, Y., Wang, W., Joty, S., & Hoi, S. C. (2021). Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. arXiv:2109.00859.

Wang, W., Wang, Y., Joty, S., & Hoi, S. C. (2023). Rap-gen: Retrieval-augmented patch generation with codet5 for automatic program repair. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 146–158.

Wei, B., Li, Y., Li, G., Xia, X., & Jin, Z. (2020). Retrieve and refine: exemplar-based neural comment generation. In: *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*, pp. 349–360.

Wei, Y., Xia, C. S., & Zhang, L. (2023). Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 172–184.

Weimer, W., Nguyen, T., Le Goues, C., & Forrest, S. (2009). Automatically finding patches using genetic programming. In: *2009 IEEE 31st International Conference on Software Engineering*, pp. 364–374. IEEE.

Wu, T., He, S., Liu, J., Sun, S., Liu, K., Han, Q.-L., & Tang, Y. (2023). A brief overview of chatgpt: The history, status quo and potential future development. *IEEE/CAA Journal of Automatica Sinica., 10*(5), 1122–1136.

Xia, C. S., & Zhang, L. (2022). Less training, more repairing please: revisiting automated program repair via zero-shot learning. In: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 959–971.

Xia, C. S., & Zhang, L. (2023). Keep the conversation going: Fixing 162 out of 337 bugs for $0.42 each using chatgpt. arXiv:2304.00385.

Xia, C. S., Ding, Y., & Zhang, L. (2023). Revisiting the plastic surgery hypothesis via large language models. arXiv:2303.10494.

Xia, C.S., Wei, Y., & Zhang, L. (2023). Automated program repair in the era of large pre-trained language models. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pp. 1482–1494. IEEE

Xia, X., Bao, L., Lo, D., Kochhar, P. S., Hassan, A. E., & Xing, Z. (2017). What do developers search for on the web? *Empirical Software Engineering, 22*, 3149–3185.

Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R. R., & Le, Q. V. (2019). Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in Neural Information Processing Systems*. *32*.

Yin, X., Ni, C., Wang, S., Li, Z., Zeng, L., & Yang, X. (2024). Thinkrepair: Self-directed automated program repair. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 1274–1286.

Zhu, Q., Sun, Z., Xiao, Y.-a., Zhang, W., Yuan, K., Xiong, Y., & Zhang, L. (2021). A syntax-guided edit decoder for neural program repair. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 341–353.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Zixin Liu is a master's student at the School of Software Engineering, Xi'an Jiaotong University, China. His research interests focus on automated program repair.

Xiaozhi Du is an Associate Professor at the School of Software Engineering, Xi'an Jiaotong University, China. He holds a Ph.D. degree, and his research interests focus on machine learning, artificial intelligence, and embedded systems.

Hairui Liu is a master's student at the School of Software Engineering, Xi'an Jiaotong University, China.